Documento de Planificación y Diseño

Proyecto: Analizador de Paquetes de Red (Sniffer)

1. Resumen

Visión General: El presente proyecto tiene como objetivo el diseño y desarrollo de una herramienta de análisis de tráfico de red (Sniffer) similar a Wireshark. Este software permitirá a administradores de sistemas, estudiantes de ciberseguridad y desarrolladores auditar y diagnosticar redes locales de manera eficiente. La arquitectura dividirá la carga de trabajo: un motor de alto rendimiento en C++ para la captura de paquetes a bajo nivel, y una interfaz gráfica moderna desarrollada en Python para la interpretación y visualización de los datos.

Objetivos:

- **Específico:** Desarrollar un analizador de paquetes para Windows que capture y decodifique tráfico de red (HTTP, TCP, UDP, ICMP) usando un núcleo en C++ y una interfaz en Python.
- **Medible:** El sistema debe ser capaz de procesar hasta 10,000 paquetes por segundo sin pérdida de datos en una red Gigabit estándar.
- **Alcanzable:** Se utilizarán librerías probadas en la industria como Npcap SDK para C++ y PyQt6/PySide para Python, asegurando la viabilidad técnica.
- **Relevante:** Proporcionará una herramienta de diagnóstico accesible, facilitando la comprensión de redes complejas mediante una UI amigable que traduce datos crudos a información legible.

2. Alcance del Proyecto

El proyecto se centrará exclusivamente en entornos de sistema operativo **Windows (10 y 11)**, enfocado en la intercepción, decodificación y análisis en tiempo real del tráfico de la red local.

Requisitos Funcionales y Técnicos:

- Detección y enumeración automática de todas las interfaces de red disponibles en el equipo anfitrión.
- Habilitación del "Modo Promiscuo" en la tarjeta de red para capturar tráfico dirigido a otras direcciones MAC en el mismo segmento.
- Filtrado en tiempo real de paquetes según el protocolo (e.g., mostrar solo HTTP o TCP).
- Inspección profunda de paquetes (Deep Packet Inspection básica) para separar las capas del modelo OSI (Enlace, Red, Transporte, Aplicación).
- Exportación e importación de sesiones de captura en el formato estándar .pcap (compatible con Wireshark).

3. Diseño y Arquitectura

La arquitectura seleccionada sigue un patrón de **Cliente-Servidor local** (o Backend-Frontend desacoplado) para maximizar el rendimiento sin comprometer la estética.

- **Motor Central (Backend - C++):** Escrito en C++ moderno, utiliza la biblioteca Npcap para interactuar directamente con los controladores del kernel de Windows. Este módulo captura los bytes crudos, extrae las cabeceras (Ethernet, IPv4/IPv6, TCP/UDP) y empaqueta la información relevante.
- **Interfaz de Usuario e Interpretación (Frontend - Python):** Desarrollado con PyQt6, proporciona un diseño oscuro/claro (Dark Mode por defecto) moderno. Python se comunica con el núcleo de C++ a través de C-bindings (Pybind11) o mediante comunicación entre procesos (Pipes/Sockets locales). Python toma los datos serializados, los formatea y los presenta en tablas y gráficos dinámicos.

Diagrama de Arquitectura de Componentes (UML)

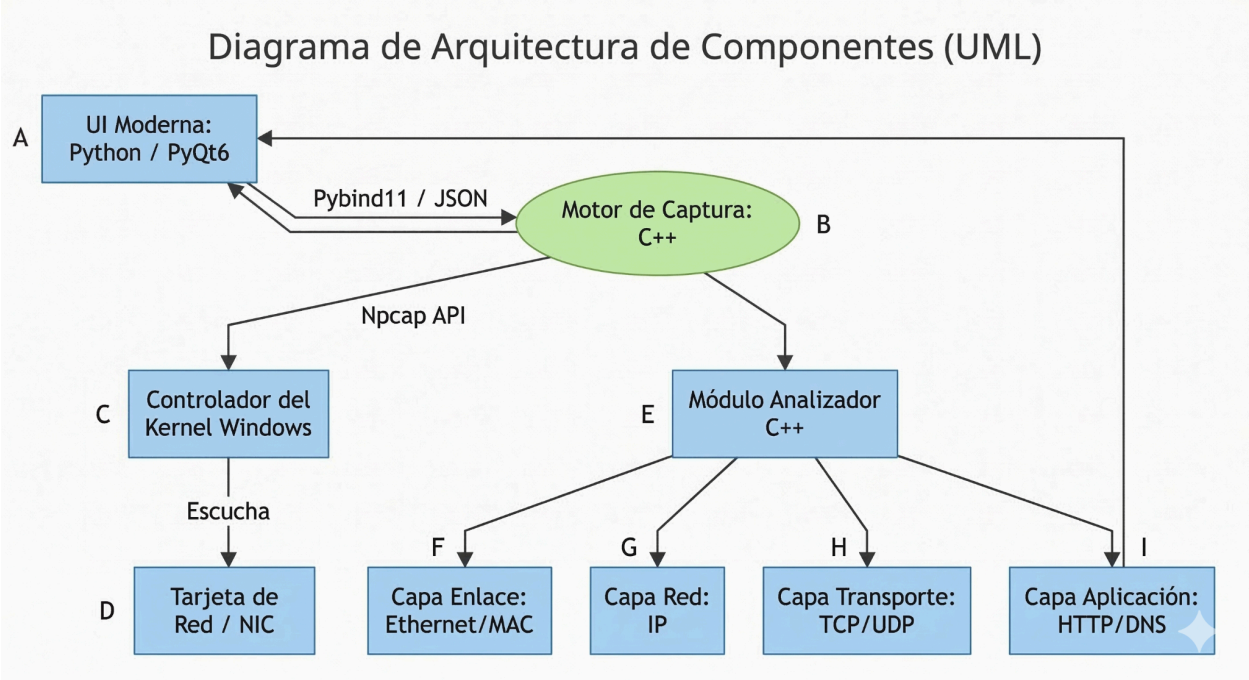
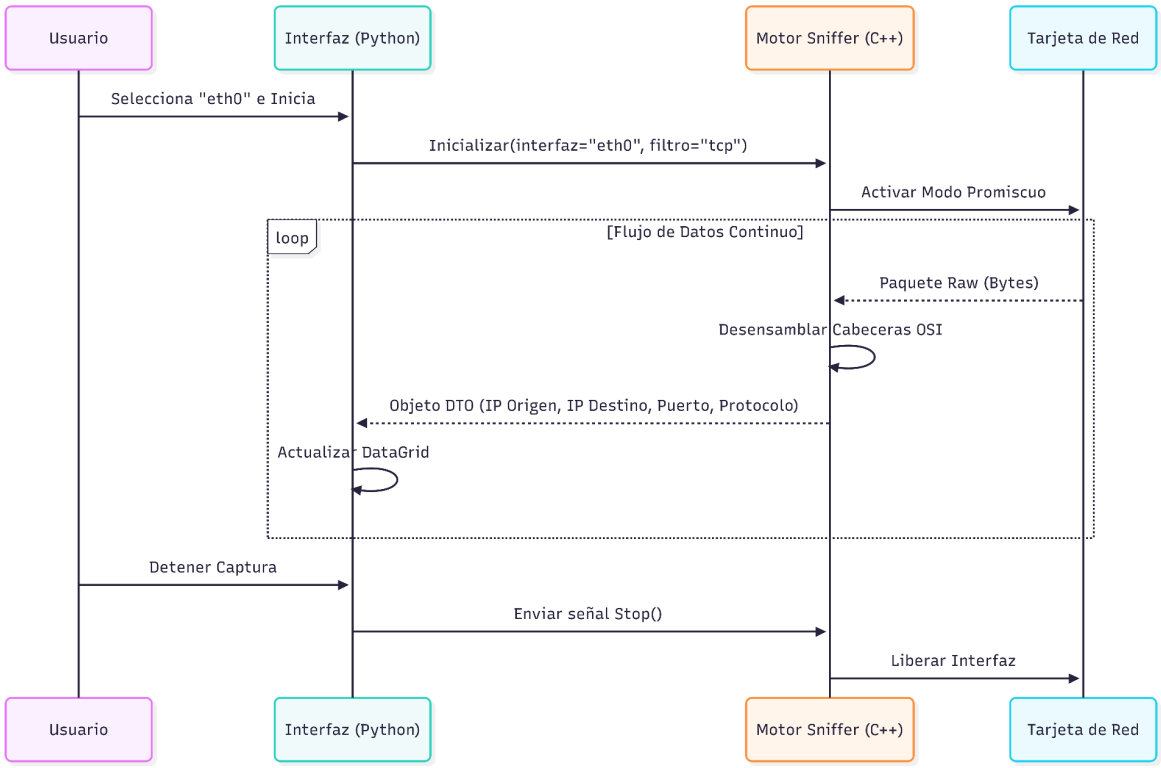


Diagrama de Secuencia de Captura (UML)



4. Cronograma del Proyecto

A continuación, se detalla el plan de ejecución dividido en 4 fases principales que abarcan los 3 meses estipulados:

Fase	Descripción de Tareas	Hitos Entregables
1. Planificación e Infraestructura	Configuración de entornos C++ y Python. Integración inicial de Pybind11 y pruebas con SDK de Npcap.	Repositorio configurado, "Hello World" entre Python y C++ funcionando.
2. Desarrollo del Motor Core (C++)	Programación del loop de captura, decodificación de protocolos base (MAC, IP, TCP, UDP),	Motor C++ capaz de imprimir logs de red en consola de manera estable.

	y manejo de hilos.	
3. Desarrollo UI y Enlace (Python)	Diseño de la interfaz gráfica con PyQt6. Implementación de tablas dinámicas, visores de detalles hexagonales e integración con C++.	Interfaz gráfica mostrando tráfico en tiempo real. Filtros básicos operacionales.
4. QA, Optimización y Empaquetado	Pruebas de estrés, corrección de fugas de memoria (memory leaks), y creación de un instalador unificado para Windows (.exe).	Versión Release Candidate 1.0 lista para distribución.

5. Recursos Necesarios

Recursos Tecnológicos:

- **Librerías y SDKs:** * *Npcap SDK:* Para la captura de paquetes nativa en Windows.
  - *Pybind11:* Para crear el puente de comunicación entre C++ y Python sin sobrecarga de red.
  - *PyQt6:* Framework para la Interfaz Gráfica (GUI).
- **Control de Versiones:** Git y GitHub/GitLab para el seguimiento del código y pipelines de CI/CD.
- **Hardware de Prueba:** Computadoras con Windows 10 y 11 que posean diversas interfaces de red (Ethernet y tarjetas Wi-Fi que soporten modo monitor/promiscuo).

6. Análisis de Riesgos

Un sistema de esta naturaleza interactúa fuertemente con los recursos del hardware y la seguridad del sistema operativo. A continuación, las estrategias de mitigación para los riesgos principales:

1. **Riesgo: Cuello de botella en el rendimiento (Pérdida de Paquetes).**
   - *Descripción:* En redes de alto tráfico (ej. descargas grandes o streaming), la UI de Python podría no procesar los datos tan rápido como C++ los captura, causando pérdida de información o congelamiento de la aplicación.

- *Mitigación:* Implementar un patrón Productor-Consumidor. El motor en C++ utilizará un búfer circular (Ring Buffer) y multithreading para almacenar paquetes temporalmente. Python consumirá estos datos en lotes (batching) cada 100ms en lugar de actualizar la pantalla paquete por paquete.

2. **Riesgo: Restricciones de Permisos en Windows.**
   - *Descripción:* Poner una tarjeta de red en modo promiscuo requiere privilegios de Administrador. Si el usuario ejecuta la aplicación de forma normal, esta fallará en silencio o se cerrará (Crash).
   - *Mitigación:* Configurar el archivo de manifiesto (Manifest) del ejecutable de Windows para que solicite elevación de privilegios (UAC prompt) automáticamente al iniciar. Además, incluir mensajes de error claros en la UI si los permisos son insuficientes.
3. **Riesgo: Falsos Positivos de Software Antivirus.**
   - *Descripción:* Las herramientas que inspeccionan tráfico de red suelen ser catalogadas como software malicioso o *Spyware* por Windows Defender u otros antivirus heurísticos.
   - *Mitigación:* Evitar el uso de técnicas de inyección de código. Firmar digitalmente los ejecutables y archivos .dll con un certificado de firma de código (Code Signing Certificate) válido. Proporcionar documentación clara a los usuarios sobre cómo agregar la aplicación a las listas de exclusión si fuese necesario durante la fase de pruebas.